javaScript

Raccolta Completa: 7 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

✨ Fonte Unica per Google NotebookLM

Indice

- Variabili
- Tipi di dato
- Operatori
- Conversioni di tipo
- Condizionali
- Cicli
- Funzioni
- Arrow function
- Scope e closures
- Hoisting

Variabili

`var` è function-scoped e soggetta a hoisting (evitare). `let` è block-scoped e riassegnabile. `const` è block-scoped e non riassegnabile (ma l'oggetto/array a cui punta è mutabile).

```
var vecchio = "evitare";        // function-scoped, hoisting
let contatore = 0;              // block-scoped, riassegnabile
const PI = 3.14159;             // block-scoped, non riassegnabile

contatore = 1;                  // OK
// PI = 3;                      // TypeError

const persona = { nome: "Alice" };
persona.nome = "Bob";           // OK: la proprietà è mutabile
persona.eta = 30;               // OK: si può aggiungere
// persona = {};                // TypeError: non si può riassegnare

const arr = [1, 2, 3];
arr.push(4);                    // OK
// arr = [];                    // TypeError
```

Tipi di dato

JavaScript ha 7 tipi primitivi e il tipo `object`.

```
// Primitivi
let stringa   = "testo";          // string
let numero    = 42;               // number
let decimale  = 3.14;             // number (non c'è un tipo float separat
let grande    = 9007199254740993n; // bigint
let booleano  = true;             // boolean
let nullo     = null;             // null (assenza intenzionale di valore)
let indefinito;                   // undefined (variabile non inizializzat
let simbolo   = Symbol("id");     // symbol (valore unico)

// Object (non primitivo)
let oggetto = { a: 1 };
let array   = [1, 2, 3];
let funzione = function() {};

// Controllo tipo
typeof "testo"       // "string"
typeof 42            // "number"
typeof 42n           // "bigint"
typeof true          // "boolean"
typeof undefined     // "undefined"
typeof Symbol()      // "symbol"
typeof null          // "object" — storico bug di JS
typeof {}            // "object"
typeof []            // "object"
typeof function(){}  // "function"

// Controllo più preciso
Array.isArray([])            // true
null === null                // true
Object.prototype.toString.call([]) // "[object Array]"
```

Operatori

```
// Aritmetici
5 + 3   // 8
5 - 3   // 2
5 * 3   // 15
5 / 3   // 1.6666...
5 % 3   // 2  (resto)
5 ** 3  // 125 (elevamento a potenza)
5 / 0   // Infinity
0 / 0   // NaN

// Incremento / decremento
let n = 5;
n++;  // 5 (restituisce prima, poi incrementa)
++n;  // 7 (incrementa prima, poi restituisce)
n--;  // 7 (restituisce prima, poi decrementa)

// Assegnazione composta
n += 10;  // n = n + 10
n -= 2;   // n = n - 2
n *= 3;   // n = n * 3
n /= 4;   // n = n / 4
n %= 5;   // n = n % 5
n **= 2;  // n = n ** 2

// Confronto
3 == "3"    // true  — uguaglianza debole (coercizione di tipo)
3 === "3"   // false — uguaglianza stretta (preferibile sempre)
3 != "3"    // false
3 !== "3"   // true
3 > 2       // true
3 >= 3      // true
3 < 4       // true
3 <= 3      // true

// Logici
true && false  // false (AND)
true || false  // true  (OR)
!true          // false (NOT)
```

```
// Short-circuit evaluation
null && "mai eseguito"    // null    (esce al primo falsy)
"a"  || "mai eseguito"    // "a"     (esce al primo truthy)
null ?? "default"         // "default" (nullish coalescing)

// Bitwise (raramente usati)
5 & 3   // 1
5 | 3   // 7
5 ^ 3   // 6
~5      // -6
5 << 1  // 10
5 >> 1  // 2

// Ternario
let msg = età >= 18 ? "adulto" : "minorenne";

// Comma operator (sconsigliato)
let x = (1, 2, 3); // x === 3

// typeof, instanceof, void, delete
typeof 42                    // "number"
[] instanceof Array          // true
void 0                       // undefined
delete obj.prop              // rimuove la proprietà, restituisce true
```

Conversioni di tipo

JavaScript esegue conversioni implicite (coercizione). Preferire conversioni esplicite.

```
// Verso stringa
String(42)        // "42"
String(true)      // "true"
String(null)      // "null"
String(undefined) // "undefined"
(42).toString()   // "42"
(255).toString(16) // "ff" (base 16)
42 + ""           // "42" (implicita, sconsigliata)

// Verso numero
Number("42")      // 42
Number("")        // 0
Number("  ")      // 0
Number("abc")     // NaN
Number(true)      // 1
Number(false)     // 0
Number(null)      // 0
Number(undefined) // NaN
parseInt("42px")  // 42
parseInt("0xff", 16) // 255
parseFloat("3.14rem") // 3.14
+"42"             // 42 (unario +, sconsigliato)

// Verso booleano
Boolean(0)          // false
Boolean("")         // false
Boolean(null)       // false
Boolean(undefined)  // false
Boolean(NaN)        // false
Boolean(false)      // false
// Tutto il resto è true
Boolean(1)          // true
Boolean("a")        // true
Boolean([])         // true  — attenzione!
Boolean({})         // true  — attenzione!
!!0                 // false (doppia negazione, sconsigliato)

// NaN
isNaN("abc")        // true (con coercizione)
Number.isNaN("abc") // false (senza coercizione, preferibile)
Number.isNaN(NaN)   // true
```

```
// Controllo valori finiti
Number.isFinite(Infinity) // false
Number.isFinite(42)       // true
Number.isInteger(42.0)    // true
Number.isInteger(42.5)    // false
```

Condizionali

```
// if / else if / else
if (punteggio >= 90) {
  console.log("A");
} else if (punteggio >= 70) {
  console.log("B");
} else {
  console.log("C");
}

// switch
switch (giorno) {
  case "lunedì":
  case "martedì":
    console.log("inizio settimana");
    break;
  case "venerdì":
    console.log("fine settimana");
    break;
  default:
    console.log("altro giorno");
}

// switch con return in una funzione (no break necessario)
function getLabel(code) {
  switch (code) {
    case 200: return "OK";
    case 404: return "Not Found";
    case 500: return "Server Error";
    default:  return "Unknown";
  }
}

// Ternario (per espressioni semplici)
const saluto = ora < 12 ? "Buongiorno" : "Buonasera";

// Guardie (early return)
function elabora(input) {
  if (!input) return null;           // esce subito
```

```
  if (typeof input !== "string") return null;
  return input.trim().toUpperCase();
}
```

Cicli

```
// for classico
for (let i = 0; i < 5; i++) {
  console.log(i); // 0 1 2 3 4
}

// for...of — itera sui valori (array, string, Map, Set, iterabili)
for (const frutto of ["mela", "pera", "uva"]) {
  console.log(frutto);
}

// for...of con indice
for (const [i, valore] of ["a", "b", "c"].entries()) {
  console.log(i, valore); // 0 "a", 1 "b", 2 "c"
}

// for...in — itera sulle chiavi enumerabili di un oggetto (usare con caut
const obj = { a: 1, b: 2 };
for (const chiave in obj) {
  console.log(chiave, obj[chiave]); // "a" 1, "b" 2
}

// while
let i = 0;
while (i < 5) {
  console.log(i++);
}

// do...while (eseguito almeno una volta)
let tentativo = 0;
do {
  tentativo++;
} while (tentativo < 3);

// break — esce dal ciclo
for (let i = 0; i < 10; i++) {
  if (i === 5) break;
}
```

```
// continue — salta all'iterazione successiva
for (let i = 0; i < 5; i++) {
  if (i === 2) continue;
  console.log(i); // 0 1 3 4
}

// label — break/continue su cicli annidati
outer: for (let i = 0; i < 3; i++) {
  for (let j = 0; j < 3; j++) {
    if (j === 1) continue outer;
    console.log(i, j);
  }
}
```

Funzioni

```
// Dichiarazione (hoisted)
function somma(a, b) {
  return a + b;
}

// Espressione (non hoisted)
const moltiplica = function(a, b) {
  return a * b;
};

// Parametri di default
function saluta(nome = "Mondo") {
  return `Ciao, ${nome}!`;
}

// Parametri rest (raccoglie gli argomenti in un array)
function sommaAll(...numeri) {
  return numeri.reduce((acc, n) => acc + n, 0);
}
sommaAll(1, 2, 3, 4); // 10

// Argomenti extra ignorati
function primiDue(a, b) { return a + b; }
primiDue(1, 2, 3, 4);  // 3

// Funzione come argomento (higher-order)
function applica(fn, valore) {
  return fn(valore);
}
applica(Math.sqrt, 16); // 4

// Funzione che restituisce funzione
function moltiplicaPer(x) {
  return function(y) {
    return x * y;
  };
}
const doppio = moltiplicaPer(2);
```

```
doppio(5); // 10

// IIFE — Immediately Invoked Function Expression
(function() {
  console.log("eseguita subito");
})();

// arguments (solo in funzioni classiche, non arrow)
function args() {
  console.log(arguments); // oggetto array-like
}
```

Arrow function

Sintassi compatta. Non ha proprio `this`, `arguments`, né `prototype`. Non può essere usata come costruttore.

```
// Sintassi
const quadrato = x => x * x;
const somma = (a, b) => a + b;
const saluta = () => "Ciao!";

// Corpo a blocco (serve il return esplicito)
const processa = (x) => {
  const risultato = x * 2;
  return risultato + 1;
};

// Restituire un oggetto (wrap tra parentesi)
const creaOggetto = (nome, eta) => ({ nome, eta });
creaOggetto("Alice", 30); // { nome: "Alice", eta: 30 }

// this lessicale — eredita il this del contesto esterno
class Timer {
  constructor() {
    this.secondi = 0;
  }
  avvia() {
    // Arrow: this è l'istanza Timer
    setInterval(() => {
      this.secondi++;
    }, 1000);
  }
}

// Differenza con funzione classica
const obj = {
  nome: "test",
  classica: function() { return this.nome; }, // "test"
  arrow: () => this.nome,                     // undefined (this è window,
};
```

Scope e closures

Lo scope determina dove una variabile è accessibile. Le closures permettono a una funzione di ricordare il proprio scope esterno.

```
// Block scope
{
  let blocco = "visibile solo qui";
  const anche = "idem";
}
// console.log(blocco); // ReferenceError

// Function scope
function esterna() {
  let x = 10;
  function interna() {
    console.log(x); // 10 — accede alla variabile esterna (closure)
  }
  interna();
}

// Closure pratica: contatore privato
function creaContatore() {
  let count = 0;
  return {
    incrementa: () => ++count,
    decrementa: () => --count,
    valore:     () => count,
  };
}
const c = creaContatore();
c.incrementa(); // 1
c.incrementa(); // 2
c.valore();     // 2

// Closure in cicli — errore classico con var
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 3 3 3 — i è condivisa
}
// Soluzione con let (block-scoped)
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 0 1 2
}
```

Hoisting

Le dichiarazioni di variabili e funzioni vengono “sollevate” in cima al loro scope prima dell’esecuzione.

```
// Funzioni dichiarate: completamente hoisted
console.log(saluta("Alice")); // "Ciao, Alice!" — funziona prima della dic
function saluta(nome) {
  return `Ciao, ${nome}!`;
}

// var: hoisted ma inizializzato a undefined
console.log(x); // undefined (non ReferenceError)
var x = 5;
console.log(x); // 5

// let/const: hoisted ma in Temporal Dead Zone — non accessibili prima
// console.log(y); // ReferenceError: Cannot access 'y' before initializat
let y = 5;

// Espressioni di funzione: non hoisted
// console.log(fn()); // TypeError: fn is not a function
const fn = () => "ciao";
```

---

html javascript

Indice

- String
- Number
- Array
- Object
- Map
- Set
- WeakMap e WeakSet
- Date
- JSON

String

Le stringhe sono immutabili. I metodi restituiscono sempre una nuova stringa.

```
const s = "  Hello, World!  ";

// Proprietà
s.length          // 18

// Accesso ai caratteri
s[0]              // " "
s.at(-1)          // " " (indice negativo: dall'ultima)
s.charAt(2)       // "H"
s.charCodeAt(2)   // 72
String.fromCharCode(72) // "H"

// Ricerca
s.includes("World")        // true
s.startsWith("  Hello")    // true
s.endsWith("!  ")          // true
s.indexOf("l")             // 4 (prima occorrenza)
s.lastIndexOf("l")         // 10 (ultima occorrenza)
s.search(/world/i)         // 9 (regex, restituisce indice)
s.match(/l+/g)             // ["ll", "l"] (tutte le occorrenze)
s.matchAll(/l/g)           // iteratore di tutti i match con info

// Trasformazione
s.toUpperCase()            // "  HELLO, WORLD!  "
s.toLowerCase()            // "  hello, world!  "
s.trim()                   // "Hello, World!" (rimuove spazi)
s.trimStart()              // "Hello, World!  "
s.trimEnd()                // "  Hello, World!"
s.replace("World", "JS")   // "  Hello, JS!  " (prima occorrenza)
s.replaceAll("l", "L")     // "  HeLLo, WorLd!  "
s.replace(/l/g, "L")       // "  HeLLo, WorLd!  " (regex globale)

// Estrazione
s.slice(2, 7)              // "Hello"
s.slice(-3)                // "  " + "!" = ultimi 3 char
s.substring(2, 7)          // "Hello" (come slice ma no indici negativi)
s.split(", ")              // ["  Hello", "World!  "]
s.split("")                // array di caratteri

// Costruzione
"ha".repeat(3)             // "hahaha"
```

```
"5".padStart(4, "0")       // "0005"
"5".padEnd(4, "0")         // "5000"
["a", "b", "c"].join("-")  // "a-b-c"

// Template literal
const nome = "Alice";
const età = 30;
`Ciao, ${nome}! Hai ${età} anni.`   // interpolazione
`${2 + 2}`                          // "4"
`${età >= 18 ? "adulto" : "minore"}` // espressione ternaria
// Multiriga
const multiriga = `
  Prima riga
  Seconda riga
`;

// Tagged template literal
function evidenzia(strings, ...values) {
  return strings.reduce((acc, str, i) =>
    acc + str + (values[i] !== undefined ? `<b>${values[i]}</b>` : ""), ""
}
evidenzia`Ciao ${nome}, hai ${età} anni`; // "Ciao <b>Alice</b>, hai <b>30<

// Caratteri speciali
"\n"   // newline
"\t"   // tab
"\\"   // backslash
"\'"   // apice
"\""   // virgolette
"\u0041" // "A" (Unicode escape)
"\u{1F600}" // emoji Unicode
```

Number

```
// Letterali
42          // intero
3.14        // decimale
1_000_000   // separatore visivo (ES2021)
0xff        // 255 in esadecimale
0o17        // 15 in ottale
0b1010      // 10 in binario
1e3         // 1000 (notazione scientifica)
1.5e-2      // 0.015

// Costanti
Number.MAX_SAFE_INTEGER   // 9007199254740991
Number.MIN_SAFE_INTEGER   // -9007199254740991
Number.MAX_VALUE          // ~1.79e+308
Number.MIN_VALUE          // ~5e-324 (più piccolo positivo)
Number.POSITIVE_INFINITY  // Infinity
Number.NEGATIVE_INFINITY  // -Infinity
Number.EPSILON            // ~2.22e-16 (più piccola differenza tra due nume
Number.NaN                // NaN

// Metodi dell'istanza
(3.14159).toFixed(2)       // "3.14" (stringa, arrotondato)
(1234.5).toFixed(0)        // "1235"
(12345.6).toExponential(2) // "1.23e+4"
(3.14159).toPrecision(4)   // "3.142"
(255).toString(16)          // "ff" (esadecimale)
(255).toString(2)           // "11111111" (binario)

// Metodi statici
Number.isNaN(NaN)          // true
Number.isFinite(42)        // true
Number.isInteger(42.0)     // true
Number.isSafeInteger(9007199254740991) // true
Number.parseInt("42px")    // 42
Number.parseFloat("3.14x") // 3.14

// Math
Math.round(4.5)    // 5
```

```
Math.ceil(4.1)     // 5
Math.floor(4.9)    // 4
Math.trunc(4.9)    // 4 (rimuove decimali senza arrotondare)
Math.abs(-5)       // 5
Math.max(1, 5, 3)  // 5
Math.min(1, 5, 3)  // 1
Math.pow(2, 10)    // 1024
Math.sqrt(16)      // 4
Math.cbrt(27)      // 3 (radice cubica)
Math.log(Math.E)   // 1
Math.log2(8)       // 3
Math.log10(1000)   // 3
Math.random()      // float casuale [0, 1)
Math.PI            // 3.14159...
Math.E             // 2.71828...
Math.sign(-5)      // -1
Math.hypot(3, 4)   // 5 (ipotenusa)
Math.clamp         // non esiste nativamente — usare: Math.min(Math.max(va

// Numero casuale in un range
function randomInt(min, max) {
  return Math.floor(Math.random() * (max - min + 1)) + min;
}
randomInt(1, 6); // dado da 1 a 6

// Precisione float — problema noto
0.1 + 0.2 === 0.3         // false
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON // true (confronto sicuro)
```

Array

Gli array sono oggetti con indici numerici. I metodi sono divisi in mutanti (modificano l'array) e non mutanti (restituiscono uno nuovo).

```
// Creazione
const a = [1, 2, 3];
const b = new Array(3);          // [empty × 3]
const c = Array.from("abc");     // ["a", "b", "c"]
const d = Array.from({length: 5}, (_, i) => i * 2); // [0, 2, 4, 6, 8]
const e = Array.of(1, 2, 3);    // [1, 2, 3]

// Proprietà
a.length  // 3

// Accesso
a[0]      // 1
a.at(-1)  // 3 (indice negativo)

// ── Metodi MUTANTI ──────────────────────────────

// Aggiunta / rimozione
a.push(4)           // aggiunge in fondo, restituisce new length → 4
a.pop()             // rimuove dall'ultimo, restituisce l'elemento
a.unshift(0)        // aggiunge in cima, restituisce new length
a.shift()           // rimuove il primo, restituisce l'elemento

// splice(start, deleteCount, ...items) — rimuove e/o inserisce
a.splice(1, 1)          // rimuove 1 elemento a indice 1
a.splice(1, 0, 99)      // inserisce 99 a indice 1 senza rimuovere
a.splice(1, 2, 10, 20)  // rimuove 2 e inserisce 10, 20

// Ordinamento
a.sort()                        // ordine lessicografico (default)
a.sort((x, y) => x - y)         // ordine numerico crescente
a.sort((x, y) => y - x)         // ordine numerico decrescente
a.sort((x, y) => x.localeCompare(y)) // ordine alfabetico locale

a.reverse()         // inverte l'array in-place
a.fill(0)           // riempie tutto con 0
a.fill(0, 2, 4)     // riempie dall'indice 2 al 3 con 0
a.copyWithin(0, 3)  // copia dal'indice 3 in poi all'inizio

// ── Metodi NON MUTANTI ──────────────────────────

// Iterazione con trasformazione
```

```
[1,2,3].map(x => x * 2)               // [2, 4, 6]
[1,2,3,4].filter(x => x % 2 === 0)    // [2, 4]
[1,2,3].reduce((acc, x) => acc + x, 0) // 6
[1,2,3].reduceRight((acc, x) => acc + x, 0) // 6 (da destra)
[[1,2],[3,4]].flat()                   // [1, 2, 3, 4]
[[1,[2]],[3]].flat(Infinity)           // [1, 2, 3] (profondità illimitata)
[1,2,3].flatMap(x => [x, x * 2])      // [1, 2, 2, 4, 3, 6]

// Ricerca
[1,2,3].find(x => x > 1)              // 2 (primo che soddisfa)
[1,2,3].findIndex(x => x > 1)         // 1
[1,2,3].findLast(x => x < 3)          // 2
[1,2,3].findLastIndex(x => x < 3)     // 1
[1,2,3].includes(2)                    // true
[1,2,3].indexOf(2)                     // 1 (-1 se non trovato)
[1,2,3].lastIndexOf(2)                 // 1

// Verifica
[1,2,3].every(x => x > 0)             // true (tutti soddisfano)
[1,2,3].some(x => x > 2)              // true (almeno uno)

// Costruzione
[1,2].concat([3,4], [5])              // [1, 2, 3, 4, 5]
[1,2,3].slice(1, 2)                   // [2]
[1,2,3].slice(-1)                     // [3]
[1,2,3].join(" - ")                   // "1 - 2 - 3"
[...new Set([1,1,2,3])].sort()        // [1, 2, 3] (deduplica)

// Conversione
Array.from(new Set([1, 2, 2, 3]))     // [1, 2, 3]
Object.entries({a:1, b:2})            // [["a",1],["b",2]]

// Iteratori
[..."a","b","c"].keys()]              // [0, 1, 2]
[..."a","b","c"].values()]            // ["a", "b", "c"]
[..."a","b","c"].entries()]           // [[0,"a"],[1,"b"],[2,"c"]]

// Metodi nuovi (ES2023+)
[1,2,3].toReversed()                   // [3, 2, 1] — non mutante
[3,1,2].toSorted((a,b) => a - b)     // [1, 2, 3] — non mutante
[1,2,3].toSpliced(1, 1, 99)          // [1, 99, 3] — non mutante
[1,2,3].with(1, 99)                    // [1, 99, 3] — sostituisce all'indi
```

Object

```
// Creazione
const persona = { nome: "Alice", eta: 30 };
const vuoto = {};
const conProto = Object.create({ saluta() { return "Ciao"; } });

// Accesso
persona.nome          // "Alice" (dot notation)
persona["eta"]        // 30 (bracket notation — utile con chiavi dinamiche
const chiave = "nome";
persona[chiave]       // "Alice"

// Modifica
persona.eta = 31;
persona.città = "Milano";     // aggiunge proprietà
delete persona.città;         // rimuove proprietà

// Verifica
"nome" in persona             // true (anche ereditato)
persona.hasOwnProperty("nome") // true (solo proprietà proprie)
Object.hasOwn(persona, "nome") // true (moderno, preferibile)

// Iterazione
Object.keys(persona)          // ["nome", "eta"]
Object.values(persona)        // ["Alice", 30]
Object.entries(persona)       // [["nome","Alice"],["eta",30]]
for (const [k, v] of Object.entries(persona)) {
  console.log(k, v);
}

// Costruzione e copia
Object.assign({}, persona, { eta: 32 }) // shallow merge
const copia = { ...persona }            // shallow clone
const merge = { ...a, ...b }            // unisce due oggetti (b sovrascri

// Creazione da entries
Object.fromEntries([["a", 1], ["b", 2]]) // { a: 1, b: 2 }
Object.fromEntries(new Map([["a", 1]]))   // { a: 1 }
```

```
// Congelamento e sigillatura
Object.freeze(persona)    // nessuna modifica, aggiunta o rimozione
Object.seal(persona)      // nessuna aggiunta/rimozione, ma modifica ok
Object.isFrozen(persona)  // true
Object.isSealed(persona)  // true

// Proprietà descrittori
Object.defineProperty(persona, "id", {
  value: 1,
  writable: false,       // non riassegnabile
  enumerable: false,     // non appare in for...in, Object.keys
  configurable: false,   // non cancellabile
});
Object.getOwnPropertyDescriptor(persona, "nome")
// { value: "Alice", writable: true, enumerable: true, configurable: true }

Object.defineProperties(persona, {
  a: { value: 1, writable: true },
  b: { value: 2, enumerable: true },
});

// Prototype
Object.getPrototypeOf(persona)    // Object.prototype
Object.setPrototypeOf(persona, null) // rimuove il prototipo (sconsigliato
```

Map

Mappa chiave→valore. Le chiavi possono essere di qualsiasi tipo (anche oggetti). Mantiene l'ordine di inserimento.

```
const map = new Map();

// Aggiunta
map.set("nome", "Alice");
map.set(42, "il numero");
map.set({ id: 1 }, "oggetto chiave");  // chiave oggetto

// Lettura
map.get("nome")     // "Alice"
map.get(42)         // "il numero"
map.has("nome")     // true
map.size            // 3

// Rimozione
map.delete("nome")  // true
map.clear()         // svuota tutto

// Iterazione
for (const [k, v] of map) { console.log(k, v); }
[...map.keys()]     // array delle chiavi
[...map.values()]   // array dei valori
[...map.entries()]  // array di [chiave, valore]
map.forEach((v, k) => console.log(k, v));

// Creazione da array
const m = new Map([["a", 1], ["b", 2]]);

// Conversione
Object.fromEntries(m)  // { a: 1, b: 2 }

// Map vs Object
// Map: qualsiasi tipo di chiave, mantiene ordine, .size, più performante
// Object: chiavi solo string/symbol, prototipo ereditato, JSON-serializza
```

Set

Collezione di valori unici. Mantiene l'ordine di inserimento.

```
const set = new Set([1, 2, 2, 3, 3]);
// Set(3) { 1, 2, 3 }

// Aggiunta / rimozione
set.add(4)       // aggiunge
set.delete(2)    // true
set.has(3)       // true
set.size         // 3
set.clear()      // svuota

// Iterazione
for (const v of set) { console.log(v); }
[...set]                  // converte in array
set.forEach(v => console.log(v));
[...set.values()]         // array dei valori
[...set.entries()]        // [[v, v], ...] (chiave = valore)

// Operazioni insiemistiche
const a = new Set([1, 2, 3]);
const b = new Set([2, 3, 4]);

const unione        = new Set([...a, ...b]);         // {1,2,3,4}
const intersezione  = new Set([...a].filter(x => b.has(x))); // {2,3}
const differenza    = new Set([...a].filter(x => !b.has(x))); // {1}
const simmetrica    = new Set([...a, ...b].filter(x => !(a.has(x) && b.has

// Deduplica array
const unici = [...new Set([1, 1, 2, 3, 3])]; // [1, 2, 3]
```

WeakMap e WeakSet

Come Map e Set, ma le chiavi (WeakMap) o i valori (WeakSet) devono essere oggetti e sono tenuti debolmente: se non ci sono altri riferimenti, il garbage collector li può rimuovere.

```
// WeakMap — per dati privati associati a oggetti
const datiPrivati = new WeakMap();
class Persona {
  constructor(nome) {
    datiPrivati.set(this, { nome });
  }
  getNome() {
    return datiPrivati.get(this).nome;
  }
}

// Metodi: get, set, has, delete (no size, no iterazione)
const wm = new WeakMap();
let obj = {};
wm.set(obj, "dato");
wm.get(obj);   // "dato"
obj = null;    // ora obj è garbage collectable

// WeakSet — per tracciare oggetti senza impedire il GC
const visitati = new WeakSet();
function visita(nodo) {
  if (visitati.has(nodo)) return;
  visitati.add(nodo);
  // processa nodo...
}

// Metodi: add, has, delete (no size, no iterazione)
```

Date

```
// Creazione
const ora = new Date();                          // adesso
const data = new Date("2024-03-15");             // da stringa ISO
const data2 = new Date(2024, 2, 15);             // anno, mese (0-based!), g
const data3 = new Date(2024, 2, 15, 10, 30);     // + ore, minuti
const ts = new Date(0);                          // epoch (1 gen 1970)

// Getter
const d = new Date("2024-03-15T10:30:00");
d.getFullYear()    // 2024
d.getMonth()       // 2 (marzo, 0-based!)
d.getDate()        // 15 (giorno del mese)
d.getDay()         // 5 (venerdì, 0=domenica)
d.getHours()       // 10
d.getMinutes()     // 30
d.getSeconds()     // 0
d.getMilliseconds() // 0
d.getTime()        // timestamp in millisecondi

// Setter
d.setFullYear(2025);
d.setMonth(0);        // gennaio
d.setDate(1);
d.setHours(12);

// Comparazione
const a = new Date("2024-01-01");
const b = new Date("2024-06-01");
a < b        // true
a.getTime() - b.getTime()  // differenza in millisecondi

// Timestamp
Date.now()            // ms dall'epoch (senza creare oggetto)

// Formattazione
d.toISOString()         // "2024-03-15T10:30:00.000Z"
d.toLocaleDateString("it-IT")  // "15/3/2024"
d.toLocaleDateString("it-IT", { day:"2-digit", month:"long", year:"numeric
```

```
// "15 marzo 2024"
d.toLocaleTimeString("it-IT")  // "10:30:00"
d.toLocaleString("it-IT")      // "15/3/2024, 10:30:00"

// Intl.DateTimeFormat (più potente)
new Intl.DateTimeFormat("it-IT", {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
}).format(d);
// "venerdì 15 marzo 2024"

// Calcoli
const domani = new Date(Date.now() + 24 * 60 * 60 * 1000);
const traUnMese = new Date(d);
traUnMese.setMonth(traUnMese.getMonth() + 1);
```

JSON

Formato di serializzazione testo. Supporta: oggetti, array, stringhe, numeri, booleani, null. Non supporta: `undefined`, funzioni, `Date`, `Map`, `Set`, `Symbol`.

```
// Serializzazione
JSON.stringify({ nome: "Alice", eta: 30 })
// '{"nome":"Alice","eta":30}'

JSON.stringify({ a: 1 }, null, 2)          // indentato con 2 spazi
JSON.stringify({ a: undefined, b: 1 })     // '{"b":1}' — undefined esclus
JSON.stringify([1, undefined, 3])          // '[1,null,3]' — undefined → nu
JSON.stringify(new Date())                 // '"2024-03-15T10:30:00.000Z"'

// Replacer
JSON.stringify({ a: 1, b: 2, c: 3 }, ["a", "c"])  // '{"a":1,"c":3}'
JSON.stringify({ a: 1, b: "testo" }, (k, v) => typeof v === "string" ? unde
// '{"a":1}'

// Deserializzazione
JSON.parse('{"nome":"Alice","eta":30}')    // { nome: "Alice", eta: 30 }
JSON.parse('[1, 2, 3]')                    // [1, 2, 3]

// Reviver
JSON.parse('{"data":"2024-03-15"}', (k, v) => k === "data" ? new Date(v) :
// { data: Date object }

// Clonazione profonda (attenzione ai limiti)
const clone = JSON.parse(JSON.stringify(obj)); // perde Date, undefined, fu

// Metodo toJSON personalizzato
class Prodotto {
  constructor(nome, prezzo) {
    this.nome = nome;
    this.prezzo = prezzo;
  }
  toJSON() {
    return { nome: this.nome, prezzoFormattato: `€ ${this.prezzo}` };
  }
}
JSON.stringify(new Prodotto("Libro", 15)); // '{"nome":"Libro","prezzoForma
```

---

html javascript

Indice

- Destructuring
- Spread e Rest
- Template literals
- Optional chaining
- Nullish coalescing
- Shorthand properties
- Computed properties
- Moduli ES6

Destructuring

Estrae valori da array o oggetti in variabili.

```
// Array destructuring
const [a, b, c] = [1, 2, 3];
const [x, , z] = [1, 2, 3];          // salta il secondo
const [primo, ...resto] = [1, 2, 3]; // rest: resto = [2, 3]
const [p = 10, q = 20] = [1];        // default: q = 20

// Swap
let m = 1, n = 2;
[m, n] = [n, m];  // m=2, n=1

// Object destructuring
const { nome, eta } = { nome: "Alice", eta: 30 };
const { nome: n2, eta: e2 } = { nome: "Alice", eta: 30 }; // rinomina
const { nome: nome3 = "Anonimo", ruolo = "user" } = {};   // con default
const { a: { b: profondi } } = { a: { b: 42 } };          // annidato

// Rest in oggetti
const { nome: n3, ...altriCampi } = { nome: "Alice", eta: 30, città: "MI" }
// altriCampi = { eta: 30, città: "MI" }

// In parametri di funzione
function mostra({ nome, eta = 0 }) {
  console.log(nome, eta);
}
mostra({ nome: "Alice", eta: 30 });

// In cicli
const persone = [{ nome: "Alice" }, { nome: "Bob" }];
for (const { nome } of persone) {
  console.log(nome);
}
```

Spread e Rest

`...` si comporta diversamente a seconda del contesto.

```
// ── Spread: espande un iterabile ──────────────────────

// In array
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5];         // [1, 2, 3, 4, 5]
const copia = [...arr1];              // shallow clone
const uniti = [...arr1, ...arr2];     // concat

// In oggetti
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 };      // { a:1, b:2, c:3 }
const override = { ...obj1, b: 99 }; // { a:1, b:99 }

// In chiamata di funzione
Math.max(...[1, 5, 3]); // 5
[].push(...[1, 2, 3]);

// Spread su stringa
[..."ciao"]  // ["c", "i", "a", "o"]

// ── Rest: raccoglie argomenti rimanenti ───────────────

// In funzione
function somma(primo, ...altri) {
  return altri.reduce((acc, n) => acc + n, primo);
}
somma(1, 2, 3, 4); // 10

// In destructuring (vedi sezione Destructuring)
const [head, ...tail] = [1, 2, 3]; // tail = [2, 3]
const { a: _, ...senzaA } = { a: 1, b: 2, c: 3 };
```

Template literals

```
const nome = "Alice";
const eta = 30;

// Interpolazione
`Ciao, ${nome}!`
`${2 + 2}`
`${eta >= 18 ? "adulto" : "minore"}`
`${[1,2,3].map(x => x*2).join(", ")}`

// Multiriga (newline reali)
const html = `
  <div class="card">
    <h2>${nome}</h2>
  </div>
`;

// Tagged template
function sql(strings, ...values) {
  const query = strings.reduce((acc, str, i) =>
    acc + str + (values[i] !== undefined ? `$${i}` : ""), "");
  return { query, params: values };
}
const id = 5;
sql`SELECT * FROM utenti WHERE id = ${id}`;
// { query: "SELECT * FROM utenti WHERE id = $1", params: [5] }

// String.raw — backslash non interpretati
String.raw`C:\Users\nome`  // "C:\\Users\\nome"
```

Optional chaining

`?.` accede a una proprietà o chiama una funzione solo se il valore a sinistra non è `null` o `undefined`. Restituisce `undefined` invece di sollevare un errore.

```
const utente = {
  nome: "Alice",
  indirizzo: { città: "Milano" },
  saluta() { return "Ciao!"; }
};

// Senza optional chaining
utente && utente.indirizzo && utente.indirizzo.città  // "Milano"

// Con optional chaining
utente?.indirizzo?.città         // "Milano"
utente?.telefono?.numero         // undefined (non lancia errore)
utente?.saluta()                 // "Ciao!"
utente?.metodoInesistente?.()    // undefined
utente?.amici?.[0]               // undefined (bracket notation)

// Con nullish coalescing
utente?.telefono?.numero ?? "N/D"  // "N/D"
```

Nullish coalescing

`??` restituisce il lato destro solo se il lato sinistro è `null` o `undefined` (non altri valori falsy come `0`, `""`, `false`).

```
null ?? "default"       // "default"
undefined ?? "default"  // "default"
0 ?? "default"          // 0      — differenza da ||
"" ?? "default"         // ""     — differenza da ||
false ?? "default"      // false  — differenza da ||

// vs OR logico
0 || "default"          // "default" (0 è falsy)
"" || "default"         // "default" ("" è falsy)

// Assegnazione nullish
let x = null;
x ??= "valore";         // x = "valore" (solo se null/undefined)
let y = 0;
y ??= "valore";         // y = 0 (non cambia: 0 non è null/undefined)

// Assegnazione logica
let a = null;
a ||= "fallback";       // a = "fallback" (se falsy)
let b = 1;
b &&= 99;               // b = 99 (se truthy)
```

Shorthand properties

```
const nome = "Alice";
const eta = 30;

// Shorthand property (ES6)
const persona = { nome, eta };  // equivale a { nome: nome, eta: eta }

// Shorthand method
const obj = {
  saluta() {
    return "Ciao!";
  },
  // equivale a: saluta: function() { ... }
};

// Computed property
const campo = "nome";
const obj2 = { [campo]: "Alice" };  // { nome: "Alice" }

// Symbol come chiave
const id = Symbol("id");
const obj3 = { [id]: 1 };
obj3[id]; // 1
```

Computed properties

```
const prefisso = "get";
const nome = "Nome";

const obj = {
  [`${prefisso}${nome}`]() {    // getMe come metodo
    return "Alice";
  },
  [`campo_${1 + 1}`]: "valore", // "campo_2": "valore"
};

obj.getNome();   // "Alice"
obj.campo_2;     // "valore"

// Utile con variabili dinamiche
function creaOggetto(chiave, valore) {
  return { [chiave]: valore };
}
creaOggetto("colore", "rosso"); // { colore: "rosso" }
```

Moduli ES6

Sistema di moduli nativo del browser (e Node.js con `.mjs` o `"type": "module"`).

```
// ── Esportazione ──────────────────────────────────

// math.js
export const PI = 3.14159;
export function somma(a, b) { return a + b; }
export class Vettore { constructor(x, y) { this.x = x; this.y = y; } }

// Export default (uno per file)
export default function moltiplica(a, b) { return a * b; }

// Rinominare in export
const interno = "valore";
export { interno as pubblico };

// Re-esportare da un altro modulo
export { somma } from "./utils.js";
export * from "./helpers.js";
export * as helpers from "./helpers.js";

// ── Importazione ──────────────────────────────────

// Importazione named
import { PI, somma } from "./math.js";

// Rinominare in import
import { somma as aggiungi } from "./math.js";

// Import default
import moltiplica from "./math.js";

// Import default + named insieme
import moltiplica, { PI, somma } from "./math.js";

// Import tutto come namespace
import * as Math2 from "./math.js";
Math2.somma(1, 2);
Math2.default(3, 4); // default export

// Import dinamico (lazy, restituisce una Promise)
async function caricaModulo() {
  const { somma } = await import("./math.js");
```

```
  return somma(1, 2);
}

// Import con assert (meta)
import data from "./dati.json" assert { type: "json" };
```

[html](#) [javascript](#)

Indice

- Callback
- Promise
- async e await
- Fetch API

Callback

Una funzione passata come argomento a un’altra funzione, chiamata al termine di un’operazione asincrona.

```
// Callback semplice
function saluta(nome, callback) {
  const messaggio = `Ciao, ${nome}!`;
  callback(messaggio);
}
saluta("Alice", (msg) => console.log(msg));

// Pattern error-first (Node.js)
fs.readFile("file.txt", "utf8", (err, data) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(data);
});

// Callback hell — problema da evitare
faiA((errA, risA) => {
  if (errA) return gestisci(errA);
  faiB(risA, (errB, risB) => {
    if (errB) return gestisci(errB);
    faiC(risB, (errC, risC) => {
      // ...sempre più annidato
    });
  });
});
```

Promise

Rappresenta il completamento (o il fallimento) di un'operazione asincrona. Tre stati: `pending`, `fulfilled`, `rejected`.

```
// Creazione
const p = new Promise((resolve, reject) => {
  const successo = true;
  if (successo) resolve("risultato");
  else          reject(new Error("errore"));
});

// Consumo
p
  .then(valore => console.log(valore))  // "risultato"
  .catch(err  => console.error(err))
  .finally(()  => console.log("finito"));

// Concatenamento (.then restituisce una nuova Promise)
fetch("/api/utente")
  .then(res  => res.json())
  .then(data => data.nome)
  .then(nome => `Ciao, ${nome}!`)
  .catch(err => console.error(err));

// Creazione rapida
Promise.resolve(42)          // Promise già risolta con 42
Promise.reject(new Error())  // Promise già rigettata

// Combinatori
Promise.all([p1, p2, p3])
// Attende tutte; rigetta se anche solo una rigetta

Promise.allSettled([p1, p2, p3])
// Attende tutte; non rigetta mai; restituisce [{status, value|reason}]

Promise.race([p1, p2, p3])
// Risolve/rigetta con la prima che si completa

Promise.any([p1, p2, p3])
// Risolve con la prima che si risolve; rigetta solo se tutte rigettano (Ag

// Esempio Promise.allSettled
const risultati = await Promise.allSettled([
  fetch("/api/a"),
  fetch("/api/b"),
```

```
]);
risultati.forEach(r => {
  if (r.status === "fulfilled") console.log(r.value);
  else console.error(r.reason);
});
```

---

async e await

Sintassi per scrivere codice asincrono in modo sincrono. Una funzione `async` restituisce sempre una Promise.

```
// Dichiarazione
async function caricaDati() {
  const risposta = await fetch("/api/dati");  // attende la Promise
  const json = await risposta.json();
  return json;
}

// Arrow async
const carica = async () => {
  const res = await fetch("/api");
  return res.json();
};

// Gestione errori
async function sicuro() {
  try {
    const res = await fetch("/api");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error("Errore:", err.message);
    return null;
  }
}

// Await in parallelo
async function parallelo() {
  // Sbagliato: sequenziale (lento)
  const a = await fetchA();
  const b = await fetchB();

  // Corretto: parallelo
  const [a2, b2] = await Promise.all([fetchA(), fetchB()]);
  return { a2, b2 };
}

// Top-level await (nei moduli ES6)
const dati = await fetch("/api/dati").then(r => r.json());

// for await...of (stream / iterabili asincroni)
async function leggiStream(stream) {
```

```
  for await (const chunk of stream) {
    console.log(chunk);
  }
}
```

Fetch API

Interfaccia per richieste HTTP. Restituisce una Promise.

```
// GET base
const res = await fetch("https://api.example.com/utenti");
const dati = await res.json();

// Verifica HTTP status
if (!res.ok) throw new Error(`Errore HTTP: ${res.status}`);

// POST con JSON
const res2 = await fetch("/api/utenti", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer TOKEN",
  },
  body: JSON.stringify({ nome: "Alice", eta: 30 }),
});

// PUT / PATCH / DELETE
await fetch(`/api/utenti/${id}`, { method: "DELETE" });
await fetch(`/api/utenti/${id}`, {
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ eta: 31 }),
});

// Form data (upload file)
const formData = new FormData();
formData.append("file", fileInput.files[0]);
formData.append("nome", "documento");
await fetch("/api/upload", { method: "POST", body: formData });

// Opzioni di fetch
fetch(url, {
  method: "GET",               // GET POST PUT PATCH DELETE HEAD OPTIONS
  headers: {},                 // oggetto o Headers
  body: null,                  // string, FormData, Blob, ArrayBuffer…
  mode: "cors",                // cors no-cors same-origin
  credentials: "same-origin",  // omit same-origin include
  cache: "default",            // no-store no-cache reload force-cache
  redirect: "follow",          // follow error manual
  referrerPolicy: "strict-origin-when-cross-origin",
```

```
  signal: controller.signal,    // AbortSignal per cancellare la richiesta
});

// AbortController — cancellare una richiesta
const controller = new AbortController();
setTimeout(() => controller.abort(), 5000); // timeout 5s
try {
  const res = await fetch("/api/lenta", { signal: controller.signal });
} catch (err) {
  if (err.name === "AbortError") console.log("Richiesta annullata");
}

// Leggere la risposta
res.json()        // Promise<object>
res.text()        // Promise<string>
res.blob()        // Promise<Blob>
res.arrayBuffer() // Promise<ArrayBuffer>
res.formData()    // Promise<FormData>
res.headers.get("Content-Type")
res.status        // 200
res.statusText    // "OK"
res.ok            // true se status 200-299
res.url           // URL finale (dopo redirect)
```

---

html javascript

Indice

- Classi
- Ereditarietà
- Getter e Setter
- Metodi statici
- Symbol
- Iteratori e generatori
- Proxy e Reflect

Classi

Sintassi ES6 per la programmazione orientata agli oggetti. Basata internamente su prototipi.

```
class Animale {
  // Campi pubblici
  verso = "...";
  #vita = 100;          // campo privato (ES2022)

  constructor(nome, specie) {
    this.nome = nome;
    this.specie = specie;
  }

  // Metodo d'istanza
  descrivi() {
    return `${this.nome} è un ${this.specie}`;
  }

  // Campo privato — accessibile solo dentro la classe
  getVita() { return this.#vita; }
  danneggia(danni) { this.#vita -= danni; }

  // Metodo statico
  static crea(nome) {
    return new Animale(nome, "generico");
  }

  // Getter / Setter
  get stato() {
    return this.#vita > 0 ? "vivo" : "morto";
  }
  set vita(v) {
    if (v < 0) throw new Error("La vita non può essere negativa");
    this.#vita = v;
  }
}

const gatto = new Animale("Micio", "felino");
gatto.descrivi();      // "Micio è un felino"
gatto.stato;           // "vivo"
gatto.vita = 80;       // usa il setter
gatto.getVita();       // 80

// Verifica
```

```
gatto instanceof Animale  // true
gatto.constructor === Animale // true
```

Ereditarietà

`extends` crea una sottoclasse. `super` chiama il costruttore o i metodi della classe padre.

```
class Cane extends Animale {
  #razza;

  constructor(nome, razza) {
    super(nome, "cane");   // chiama il costruttore di Animale
    this.#razza = razza;
    this.verso = "Bau!";
  }

  abbaia() {
    return `${this.nome}: ${this.verso}`;
  }

  // Override
  descrivi() {
    return `${super.descrivi()} di razza ${this.#razza}`;
  }
}

const rex = new Cane("Rex", "Labrador");
rex.descrivi();   // "Rex è un cane di razza Labrador"
rex.abbaia();     // "Rex: Bau!"
rex instanceof Cane    // true
rex instanceof Animale // true

// Mixin — ereditarietà multipla simulata
const Nuotatore = (Base) => class extends Base {
  nuota() { return `${this.nome} nuota`; }
};
const Volatore = (Base) => class extends Base {
  vola() { return `${this.nome} vola`; }
};

class Anatra extends Nuotatore(Volatore(Animale)) {}
const paperino = new Anatra("Paperino", "anatra");
paperino.nuota(); // "Paperino nuota"
paperino.vola();  // "Paperino vola"
```

Getter e Setter

Permettono di intercettare la lettura e la scrittura di una proprietà.

```
class Temperatura {
  #celsius;

  constructor(celsius) {
    this.#celsius = celsius;
  }

  get celsius()    { return this.#celsius; }
  set celsius(v)   {
    if (v < -273.15) throw new RangeError("Sotto lo zero assoluto");
    this.#celsius = v;
  }

  get fahrenheit() { return this.#celsius * 9/5 + 32; }
  set fahrenheit(v){ this.#celsius = (v - 32) * 5/9; }

  get kelvin()     { return this.#celsius + 273.15; }
}

const t = new Temperatura(100);
t.fahrenheit;       // 212
t.fahrenheit = 32;
t.celsius;          // 0

// Object.defineProperty
const obj = {};
Object.defineProperty(obj, "sola_lettura", {
  get() { return 42; },
  enumerable: true,
  configurable: false,
});
obj.sola_lettura; // 42
```

Metodi statici

Appartengono alla classe, non alle istanze. Utili per factory, utility e costanti.

```
class MathUtils {
  static PI = 3.14159;

  static somma(a, b)  { return a + b; }
  static clamp(v, min, max) { return Math.min(Math.max(v, min), max); }

  static #privata()   { return "solo dentro la classe"; }
}

MathUtils.somma(1, 2);   // 3
MathUtils.PI;             // 3.14159

// Ereditarietà dei metodi statici
class EstesaMath extends MathUtils {}
EstesaMath.somma(1, 2);  // 3
EstesaMath.PI;            // 3.14159
```

Symbol

Tipo primitivo che produce un valore unico e immutabile. Usato come chiave di proprietà per evitare collisioni.

```
const id = Symbol("id");
const id2 = Symbol("id");
id === id2   // false — ogni Symbol è unico

// Come chiave di proprietà
const utente = {
  nome: "Alice",
  [id]: 42,
};
utente[id];  // 42
Object.keys(utente);    // ["nome"] — Symbol non enumerato
Object.getOwnPropertySymbols(utente); // [Symbol(id)]

// Symbol globale (registro condiviso)
const s1 = Symbol.for("app.id");
const s2 = Symbol.for("app.id");
s1 === s2  // true
Symbol.keyFor(s1);  // "app.id"

// Symbol well-known (personalizzare comportamenti)
class Lista {
  constructor(...items) { this.items = items; }

  [Symbol.iterator]() {         // rende l'oggetto iterabile
    let i = 0;
    return { next: () => ({ value: this.items[i], done: i++ >= this.items.l
  }

  get [Symbol.toStringTag]() {  // personalizza Object.prototype.toString
    return "Lista";
  }
}

for (const item of new Lista(1, 2, 3)) { console.log(item); }
Object.prototype.toString.call(new Lista()); // "[object Lista]"
```

Iteratori e generatori

Un iteratore è un oggetto con `next()`. Un generatore è una funzione che produce iteratori usando `yield`.

```
// Iteratore manuale
function creaRange(start, end) {
  let current = start;
  return {
    next() {
      if (current <= end)
        return { value: current++, done: false };
      return { value: undefined, done: true };
    },
    [Symbol.iterator]() { return this; }
  };
}
for (const n of creaRange(1, 3)) { console.log(n); } // 1 2 3

// Generatore
function* genera() {
  yield 1;
  yield 2;
  yield 3;
}
const gen = genera();
gen.next(); // { value: 1, done: false }
gen.next(); // { value: 2, done: false }
gen.next(); // { value: 3, done: false }
gen.next(); // { value: undefined, done: true }

// Range con generatore
function* range(start, end, step = 1) {
  for (let i = start; i <= end; i += step) yield i;
}
[...range(1, 10, 2)]; // [1, 3, 5, 7, 9]

// Generatore infinito
function* naturali() {
  let n = 0;
  while (true) yield n++;
}
const gen2 = naturali();
gen2.next().value; // 0
gen2.next().value; // 1
```

```
// yield*: delega a un altro iterabile
function* concatena(...arrays) {
  for (const arr of arrays) yield* arr;
}
[...concatena([1,2],[3,4])]; // [1, 2, 3, 4]

// Generatore asincrono
async function* fetchPagine(url) {
  let pagina = 1;
  while (true) {
    const res = await fetch(`${url}?page=${pagina++}`);
    if (!res.ok) break;
    yield await res.json();
  }
}
for await (const pagina of fetchPagine("/api/items")) {
  console.log(pagina);
}
```

Proxy e Reflect

`Proxy` intercetta operazioni su un oggetto. `Reflect` fornisce i metodi di default per le stesse operazioni.

```
const handler = {
  get(target, prop, receiver) {
    console.log(`Letto: ${prop}`);
    return Reflect.get(target, prop, receiver);
  },
  set(target, prop, value, receiver) {
    if (typeof value !== "number") throw new TypeError("Solo numeri");
    console.log(`Scritto: ${prop} = ${value}`);
    return Reflect.set(target, prop, value, receiver);
  },
  has(target, prop) {
    console.log(`Verificato: ${prop} in obj`);
    return Reflect.has(target, prop);
  },
  deleteProperty(target, prop) {
    console.log(`Cancellato: ${prop}`);
    return Reflect.deleteProperty(target, prop);
  },
  apply(target, thisArg, args) {
    console.log(`Chiamato con: ${args}`);
    return Reflect.apply(target, thisArg, args);
  },
};

const obj = new Proxy({ x: 1 }, handler);
obj.x;         // "Letto: x" → 1
obj.y = 42;    // "Scritto: y = 42"
"x" in obj;    // "Verificato: x in obj"

// Validazione automatica
function creaValidato(schema) {
  return new Proxy({}, {
    set(target, prop, value) {
      if (schema[prop] && typeof value !== schema[prop])
        throw new TypeError(`${prop} deve essere ${schema[prop]}`);
      target[prop] = value;
      return true;
    }
  });
}
const p = creaValidato({ nome: "string", eta: "number" });
```

```
p.nome = "Alice"; // OK
p.eta = "trenta"; // TypeError
```

[html](#) [javascript](#)

Indice

- try catch finally
- Tipi di errore
- Errori custom

try catch finally

Gestione degli errori a runtime.

```
try {
  // codice che potrebbe lanciare un errore
  const dati = JSON.parse(testo);
  if (!dati) throw new Error("Dati non validi");
} catch (err) {
  // err è l'oggetto errore
  console.error(err.name);     // "SyntaxError"
  console.error(err.message);  // messaggio
  console.error(err.stack);    // stack trace
} finally {
  // eseguito sempre, con o senza errore
  console.log("Fine elaborazione");
}

// Catch opzionale (binding opzionale ES2019)
try {
  rischiosa();
} catch {
  console.log("qualcosa è andato storto");
}

// Re-throw — rilancia solo errori inattesi
try {
  esegui();
} catch (err) {
  if (err instanceof ValidationError) {
    mostraMessaggio(err.message);
  } else {
    throw err; // rilancia altri errori
  }
}
```

Tipi di errore

```
// Errori nativi
new Error("messaggio generico")
new TypeError("tipo non corretto")         // tipo sbagliato
new RangeError("fuori dal range")          // valore fuori range
new ReferenceError("variabile non definita") // variabile inesistente
new SyntaxError("JSON malformato")         // sintassi non valida
new URIError("URI malformato")             // decodeURIComponent di stringa
new EvalError("errore di eval")            // errore in eval() (raro)
new AggregateError([e1, e2], "più errori") // Promise.any

// Proprietà degli errori
const err = new TypeError("valore atteso number");
err.name;     // "TypeError"
err.message;  // "valore atteso number"
err.stack;    // stack trace come stringa
err.cause;    // errore originale (se passato in options)

// Error cause (ES2022)
try {
  connetti();
} catch (cause) {
  throw new Error("Connessione fallita", { cause });
}
```

Errori custom

```
class AppError extends Error {
  constructor(message, code, details = {}) {
    super(message);
    this.name = "AppError";
    this.code = code;
    this.details = details;
    // Mantiene lo stack corretto in V8
    if (Error.captureStackTrace) {
      Error.captureStackTrace(this, AppError);
    }
  }
}

class ValidationError extends AppError {
  constructor(campo, messaggio) {
    super(messaggio, "VALIDATION_ERROR", { campo });
    this.name = "ValidationError";
    this.campo = campo;
  }
}

class NotFoundError extends AppError {
  constructor(risorsa, id) {
    super(`${risorsa} con id ${id} non trovato`, "NOT_FOUND");
    this.name = "NotFoundError";
  }
}

// Uso
try {
  throw new ValidationError("email", "Formato email non valido");
} catch (err) {
  if (err instanceof ValidationError) {
    console.log(err.campo, err.message);
  }
}
```

html javascript

Indice

- Selezione elementi
- Modifica elementi
- Creazione e rimozione nodi
- Classi e stili
- Attributi
- Traversal del DOM
- Eventi
- Event delegation
- MutationObserver
- IntersectionObserver
- ResizeObserver
- localStorage e sessionStorage
- Cookies
- URL e History API
- setTimeout e setInterval
- requestAnimationFrame
- Clipboard API
- Geolocation API
- Notification API
- Web Workers
- Console
- Clonazione oggetti
- Debounce e throttle
- Currying
- Memoization

Selezione elementi

```
// Selettori moderni (restituiscono null se non trovato)
document.querySelector(".card")              // primo elemento
document.querySelector("#titolo h2")         // selettore CSS complesso
document.querySelectorAll(".card")           // NodeList (statica)
document.querySelectorAll("a[href^='https']") // attributo

// Selettori classici
document.getElementById("main")              // Element | null
document.getElementsByClassName("card")      // HTMLCollection (live)
document.getElementsByTagName("p")           // HTMLCollection (live)
document.getElementsByName("email")          // NodeList

// Dal contesto di un elemento
const container = document.querySelector(".container");
container.querySelector(".item")             // solo dentro container
container.querySelectorAll(".item")

// Controllo e cast
const el = document.querySelector(".card");
el instanceof HTMLElement  // true
el.matches(".card.attiva") // true/false
el.closest(".wrapper")     // primo antenato che corrisponde al selettore
```

Modifica elementi

```
const el = document.querySelector(".card");

// Testo
el.textContent = "Nuovo testo";          // solo testo, sicuro (no HTML)
el.textContent;                           // legge il testo (no tag)
el.innerText;                             // testo visibile (considera CSS

// HTML
el.innerHTML = "<strong>Bold</strong>";  // interpreta HTML (attenzione XSS
el.innerHTML;                             // legge l'HTML interno
el.outerHTML;                             // legge l'HTML dell'elemento + c

// Metodo sicuro contro XSS
el.textContent = inputUtente;            // mai innerHTML con input utente

// Inserimento HTML sicuro (moderno)
el.insertAdjacentHTML("beforebegin", "<p>Prima dell'elemento</p>");
el.insertAdjacentHTML("afterbegin",  "<p>Primo figlio</p>");
el.insertAdjacentHTML("beforeend",   "<p>Ultimo figlio</p>");
el.insertAdjacentHTML("afterend",    "<p>Dopo l'elemento</p>");
el.insertAdjacentText("beforeend",   "testo sicuro");

// Proprietà specifiche degli input
const input = document.querySelector("input");
input.value;               // valore corrente
input.value = "nuovo";
input.checked;             // per checkbox/radio
input.disabled = true;
input.focus();
input.blur();
input.select();            // seleziona il testo
```

Creazione e rimozione nodi

```
// Creazione
const div   = document.createElement("div");
const testo = document.createTextNode("Ciao");
const clone = div.cloneNode(true);  // true = deep clone (con figli)

// Aggiunta
document.body.appendChild(div);           // in fondo al body
document.body.prepend(div);               // all'inizio del body
parent.insertBefore(div, riferimento);    // prima di un nodo
parent.append(div, "testo", altroEl);     // uno o più nodi/stringhe
parent.prepend("testo", div);

// Sostituzione e rimozione
parent.replaceChild(nuovo, vecchio);
el.replaceWith(nuovo);                    // sostituisce l'elemento stesso
el.remove();                              // rimuove l'elemento

// insertAdjacentElement
el.insertAdjacentElement("beforebegin", nuovo);
el.insertAdjacentElement("afterend", nuovo);

// DocumentFragment — minimizza i reflow
const frag = document.createDocumentFragment();
for (let i = 0; i < 100; i++) {
  const li = document.createElement("li");
  li.textContent = `Item ${i}`;
  frag.appendChild(li);
}
document.querySelector("ul").appendChild(frag); // un solo reflow
```

Classi e stili

```
const el = document.querySelector(".card");

// classList
el.classList.add("attiva");
el.classList.add("grande", "visibile");    // più classi
el.classList.remove("attiva");
el.classList.toggle("aperto");             // aggiunge se assente, rimuove
el.classList.toggle("aperto", condizione); // forza aggiunta/rimozione
el.classList.contains("attiva");           // true/false
el.classList.replace("vecchia", "nuova");
el.className;                              // stringa con tutte le classi
el.className = "nuova lista classi";

// Stili inline
el.style.color = "red";
el.style.backgroundColor = "blue";         // camelCase
el.style.fontSize = "1.5rem";
el.style.cssText = "color: red; font-size: 1rem;"; // più stili in una vol
el.style.removeProperty("color");

// Stili calcolati (include CSS, non solo inline)
const computed = getComputedStyle(el);
computed.color;              // "rgb(255, 0, 0)"
computed.getPropertyValue("font-size"); // "24px"

// CSS custom properties (variabili CSS)
el.style.setProperty("--colore-testo", "#333");
getComputedStyle(el).getPropertyValue("--colore-testo"); // "#333"
document.documentElement.style.setProperty("--primario", "#4f46e5"); // gl
```

Attributi

```
const el = document.querySelector("a");

// getAttribute / setAttribute
el.getAttribute("href")           // valore come stringa
el.setAttribute("href", "/nuova") // imposta attributo
el.removeAttribute("target")      // rimuove
el.hasAttribute("disabled")       // true/false
el.toggleAttribute("disabled")    // aggiunge se assente, rimuove se prese

// Proprietà riflesse (più comodo per attributi standard)
el.href           // URL assoluto (diverso da getAttribute)
el.id
el.className
el.src
el.alt
el.disabled = true
el.checked = true

// dataset — attributi data-*
// <div data-utente-id="42" data-ruolo="admin">
el.dataset.utenteId  // "42" (camelCase automatico)
el.dataset.ruolo     // "admin"
el.dataset.nuovo = "valore"    // aggiunge data-nuovo="valore"
delete el.dataset.ruolo        // rimuove data-ruolo

// Attributi booleani
el.setAttribute("disabled", "")  // aggiunge disabled
el.removeAttribute("disabled")   // rimuove disabled
// equivalente a:
el.disabled = true;
el.disabled = false;
```

Traversal del DOM

```
const el = document.querySelector(".card");

// Genitori
el.parentElement           // genitore (Element)
el.parentNode              // genitore (Node, può essere document)
el.closest(".wrapper")     // antenato più vicino che corrisponde al selet

// Figli
el.children                // HTMLCollection dei figli Element
el.childNodes              // NodeList di tutti i nodi (include testo, comm
el.firstElementChild       // primo figlio Element
el.lastElementChild        // ultimo figlio Element
el.childElementCount       // numero di figli Element

// Fratelli
el.nextElementSibling      // fratello successivo Element
el.previousElementSibling  // fratello precedente Element
el.nextSibling             // nodo successivo (include testo)
el.previousSibling         // nodo precedente (include testo)

// Dimensioni e posizione
el.getBoundingClientRect()
// { top, left, right, bottom, width, height, x, y }
el.offsetWidth             // larghezza incluso padding, no margin
el.offsetHeight
el.clientWidth             // larghezza incluso padding, no border
el.scrollWidth             // larghezza totale incluso overflow
el.scrollTop               // scroll verticale del contenuto
el.scrollLeft
window.scrollY             // scroll verticale della pagina
window.scrollX
el.scrollIntoView({ behavior: "smooth", block: "start" })
```

Eventi

```
const btn = document.querySelector("#btn");

// Aggiunta
btn.addEventListener("click", handler);
btn.addEventListener("click", handler, { once: true });     // una sola vo
btn.addEventListener("click", handler, { passive: true });  // per scroll
btn.addEventListener("click", handler, { capture: true });  // fase di cat

// Rimozione (lo stesso handler, non una funzione anonima)
btn.removeEventListener("click", handler);

// Oggetto evento
function handler(event) {
  event.type;             // "click"
  event.target;           // elemento che ha scatenato l'evento
  event.currentTarget;    // elemento su cui è il listener
  event.preventDefault(); // blocca il comportamento default (es. submit, 
  event.stopPropagation(); // blocca il bubbling
  event.stopImmediatePropagation(); // blocca anche i listener successivi

  // Mouse
  event.clientX / event.clientY    // rispetto al viewport
  event.pageX / event.pageY        // rispetto alla pagina
  event.offsetX / event.offsetY    // rispetto all'elemento
  event.button    // 0=sinistro 1=centrale 2=destro
  event.buttons   // bitmask dei tasti premuti
  event.altKey / event.ctrlKey / event.shiftKey / event.metaKey

  // Tastiera
  event.key       // "Enter", "ArrowUp", "a", "A"…
  event.code      // "KeyA", "Enter", "Space"…
  event.repeat    // true se tasto tenuto premuto
  event.isComposing // true durante input IME

  // Touch
  event.touches           // lista dei touch attivi
  event.changedTouches    // touch cambiati in questo evento
  event.targetTouches     // touch sull'elemento target
```

```
  // Form / input
  event.data              // carattere inserito (InputEvent)
  event.inputType         // "insertText", "deleteContentBackward"…
}

// Tipi di evento comuni
"click" "dblclick" "mousedown" "mouseup" "mousemove"
"mouseenter" "mouseleave" "mouseover" "mouseout"
"contextmenu"
"keydown" "keyup" "keypress"  // keypress deprecato
"focus" "blur" "focusin" "focusout"
"input" "change" "submit" "reset"
"scroll" "resize"
"load" "DOMContentLoaded" "unload" "beforeunload"
"touchstart" "touchmove" "touchend" "touchcancel"
"pointerdown" "pointermove" "pointerup"
"dragstart" "drag" "dragover" "drop" "dragend" "dragenter" "dragleave"
"transitionstart" "transitionend"
"animationstart" "animationend" "animationiteration"
"copy" "cut" "paste"
"online" "offline"
"popstate" "hashchange"
"error" "abort"
"visibilitychange"
"fullscreenchange"

// Evento custom
const evento = new CustomEvent("mioEvento", {
  detail: { dati: "payload" },
  bubbles: true,
  cancelable: true,
});
el.dispatchEvent(evento);
el.addEventListener("mioEvento", e => console.log(e.detail));
```

Event delegation

Invece di mettere un listener su ogni figlio, se ne mette uno sul genitore sfruttando il bubbling.

```
const lista = document.querySelector("ul");

lista.addEventListener("click", (event) => {
  const item = event.target.closest("li");  // trova il <li> più vicino
  if (!item || !lista.contains(item)) return;

  const id = item.dataset.id;
  console.log("Cliccato item:", id);
});

// Aggiungendo elementi dinamici non serve aggiungere listener
const nuovoItem = document.createElement("li");
nuovoItem.textContent = "Nuovo";
nuovoItem.dataset.id = "99";
lista.appendChild(nuovoItem); // gestito automaticamente
```

MutationObserver

Osserva le modifiche al DOM senza polling.

```
const observer = new MutationObserver((mutations) => {
  for (const mutation of mutations) {
    if (mutation.type === "childList") {
      mutation.addedNodes.forEach(n => console.log("Aggiunto:", n));
      mutation.removedNodes.forEach(n => console.log("Rimosso:", n));
    }
    if (mutation.type === "attributes") {
      console.log(`Attributo "${mutation.attributeName}" cambiato`);
      console.log("Valore precedente:", mutation.oldValue);
    }
    if (mutation.type === "characterData") {
      console.log("Testo cambiato");
    }
  }
});

observer.observe(document.body, {
  childList: true,           // nodi figli aggiunti/rimossi
  subtree: true,             // osserva tutti i discendenti
  attributes: true,          // cambiamenti agli attributi
  attributeOldValue: true,   // salva il valore precedente
  attributeFilter: ["class", "id"], // filtra attributi specifici
  characterData: true,       // cambiamenti al testo
  characterDataOldValue: true,
});

observer.disconnect(); // smette di osservare
const pending = observer.takeRecords(); // ottiene mutation non processate
```

IntersectionObserver

Osserva quando un elemento entra o esce dal viewport (o da un altro elemento).

```
const observer = new IntersectionObserver((entries) => {
  for (const entry of entries) {
    if (entry.isIntersecting) {
      entry.target.classList.add("visibile");
      console.log("Visibilità:", entry.intersectionRatio);
    }
  }
}, {
  root: null,               // null = viewport
  rootMargin: "0px 0px -100px 0px", // margini del root
  threshold: [0, 0.5, 1],  // callback al 0%, 50%, 100% di visibilità
});

document.querySelectorAll(".animato").forEach(el => observer.observe(el));
observer.unobserve(el);   // smette di osservare un elemento
observer.disconnect();    // smette di osservare tutti

// Lazy loading immagini
const imgObserver = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      imgObserver.unobserve(img);
    }
  });
});
document.querySelectorAll("img[data-src]").forEach(img => imgObserver.obse
```

ResizeObserver

Osserva i cambiamenti di dimensione di un elemento.

```
const observer = new ResizeObserver((entries) => {
  for (const entry of entries) {
    const { width, height } = entry.contentRect;
    console.log(`${entry.target.id}: ${width}×${height}`);

    // Box size (considera padding e border)
    const [box] = entry.borderBoxSize;
    console.log("Border box:", box.inlineSize, box.blockSize);
  }
});

observer.observe(document.querySelector(".container"));
observer.observe(el, { box: "border-box" }); // "content-box" (default) o
observer.unobserve(el);
observer.disconnect();
```

---

localStorage e sessionStorage

Archiviazione dati nel browser. `localStorage` persiste tra sessioni, `sessionStorage` solo nella scheda corrente.

```
// localStorage
localStorage.setItem("tema", "scuro");
localStorage.getItem("tema");           // "scuro"
localStorage.removeItem("tema");
localStorage.clear();                   // rimuove tutto
localStorage.length;                    // numero di chiavi
localStorage.key(0);                    // prima chiave

// Oggetti (serializzare in JSON)
const utente = { nome: "Alice", eta: 30 };
localStorage.setItem("utente", JSON.stringify(utente));
const salvato = JSON.parse(localStorage.getItem("utente") ?? "null");

// sessionStorage — stessa API di localStorage
sessionStorage.setItem("passo", "2");
sessionStorage.getItem("passo");   // "2"

// Evento storage (cross-tab, solo localStorage)
window.addEventListener("storage", (event) => {
  console.log(event.key);        // chiave modificata
  console.log(event.oldValue);   // valore precedente
  console.log(event.newValue);   // nuovo valore
  console.log(event.url);        // URL della tab che ha modificato
  console.log(event.storageArea); // localStorage o sessionStorage
});
```

Cookies

```
// Scrittura
document.cookie = "nome=Alice";
document.cookie = "tema=scuro; path=/; max-age=86400"; // scade in 1 giorn
document.cookie = "sessione=token; path=/; secure; samesite=Strict"; // so
document.cookie = `data=${encodeURIComponent(JSON.stringify({a:1}))}`;

// Attributi
// expires=DATE   — data scadenza (formato UTC)
// max-age=N      — secondi dalla creazione
// path=/         — percorso valido
// domain=.example.com — dominio e sottodomini
// secure         — solo HTTPS
// samesite=Strict|Lax|None — protezione CSRF
// httponly       — non accessibile via JS (solo server)

// Lettura (tutti in una stringa)
document.cookie; // "nome=Alice; tema=scuro"

// Parsing
function getCookie(nome) {
  return Object.fromEntries(
    document.cookie.split("; ").map(c => c.split("=").map(decodeURICompone
  )[nome] ?? null;
}
getCookie("tema"); // "scuro"

// Eliminazione (impostare scadenza nel passato)
document.cookie = "nome=; max-age=0; path=/";
```

URL e History API

```
// URL API
const url = new URL("https://example.com/percorso?q=test&pagina=2#sezione")
url.protocol     // "https:"
url.hostname     // "example.com"
url.host         // "example.com" (con porta se non standard)
url.port         // ""
url.pathname     // "/percorso"
url.search       // "?q=test&pagina=2"
url.hash         // "#sezione"
url.href         // URL completo

// URLSearchParams
url.searchParams.get("q")          // "test"
url.searchParams.get("pagina")     // "2"
url.searchParams.set("pagina", "3")
url.searchParams.append("filtro", "nuovo")
url.searchParams.delete("q")
url.searchParams.has("pagina")     // true
url.searchParams.toString()        // "pagina=3&filtro=nuovo"
for (const [k, v] of url.searchParams) { console.log(k, v); }

// History API
history.pushState({ id: 1 }, "", "/nuova-url");   // aggiunge voce
history.replaceState({ id: 2 }, "", "/altra-url"); // sostituisce voce cor
history.back();       // indietro
history.forward();    // avanti
history.go(-2);       // 2 passi indietro
history.length;       // numero di voci nella history

// Evento popstate (tasto back/forward del browser)
window.addEventListener("popstate", (event) => {
  console.log("State:", event.state);
  console.log("URL:", location.href);
});

// location
location.href      // URL corrente
location.pathname  // "/percorso"
```

```
location.reload()  // ricarica la pagina
location.assign("/nuova")   // naviga
location.replace("/nuova")  // naviga senza aggiungere alla history
```

---

setTimeout e setInterval

```
// setTimeout — esegue una volta dopo N ms
const id = setTimeout(() => {
  console.log("Eseguito dopo 2 secondi");
}, 2000);
clearTimeout(id); // annulla

// setInterval — esegue ogni N ms
const idInterval = setInterval(() => {
  console.log("Ogni secondo");
}, 1000);
clearInterval(idInterval); // ferma

// setTimeout con argomenti
setTimeout((a, b) => console.log(a + b), 500, 1, 2);

// Timeout "zero" — mette in fondo alla coda degli eventi
setTimeout(() => console.log("dopo"), 0);
console.log("prima"); // "prima" → "dopo"

// Intervallo accurato con setTimeout ricorsivo
function eseguiOgni(ms, callback) {
  let id;
  function loop() {
    callback();
    id = setTimeout(loop, ms);
  }
  id = setTimeout(loop, ms);
  return () => clearTimeout(id); // restituisce funzione di stop
}

// Promise con timeout
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
await delay(1000); // attende 1 secondo
```

requestAnimationFrame

Schedula una callback prima del prossimo frame del browser. Ideale per animazioni.

```
let start = null;

function anima(timestamp) {
  if (!start) start = timestamp;
  const progresso = timestamp - start;
  const percentuale = Math.min(progresso / 1000, 1); // 1 secondo

  elemento.style.opacity = percentuale;

  if (percentuale < 1) {
    requestAnimationFrame(anima);
  }
}

const id = requestAnimationFrame(anima);
cancelAnimationFrame(id); // annulla

// Loop di animazione
let running = true;
function loop(ts) {
  if (!running) return;
  // disegna frame...
  requestAnimationFrame(loop);
}
requestAnimationFrame(loop);
```

Clipboard API

```
// Copia testo (richiede gesto utente o permesso)
await navigator.clipboard.writeText("testo copiato");

// Leggi testo
const testo = await navigator.clipboard.readText();

// Copia contenuto ricco (HTML, immagini)
await navigator.clipboard.write([
  new ClipboardItem({
    "text/plain": new Blob(["testo"], { type: "text/plain" }),
    "text/html":  new Blob(["<b>testo</b>"], { type: "text/html" }),
  })
]);

// Metodo classico (fallback per browser vecchi)
function copiaClassico(testo) {
  const el = document.createElement("textarea");
  el.value = testo;
  el.style.position = "fixed";
  el.style.opacity = "0";
  document.body.appendChild(el);
  el.select();
  document.execCommand("copy"); // deprecato ma compatibile
  document.body.removeChild(el);
}
```

Geolocation API

```
// Posizione singola
navigator.geolocation.getCurrentPosition(
  (pos) => {
    console.log(pos.coords.latitude);
    console.log(pos.coords.longitude);
    console.log(pos.coords.accuracy);     // metri
    console.log(pos.coords.altitude);     // null se non disponibile
    console.log(pos.coords.speed);        // m/s
    console.log(pos.coords.heading);      // gradi (0=Nord)
    console.log(pos.timestamp);           // ms dall'epoch
  },
  (err) => {
    console.error(err.code, err.message);
    // 1: PERMISSION_DENIED
    // 2: POSITION_UNAVAILABLE
    // 3: TIMEOUT
  },
  {
    enableHighAccuracy: true,  // GPS se disponibile
    timeout: 10000,            // ms
    maximumAge: 60000,         // usa cache se < 60s
  }
);

// Posizione continua
const watchId = navigator.geolocation.watchPosition(callback, errCallback,
navigator.geolocation.clearWatch(watchId);
```

Notification API

```
// Richiesta permesso
const permesso = await Notification.requestPermission();
// "granted" | "denied" | "default"

// Creazione notifica
if (permesso === "granted") {
  const notifica = new Notification("Titolo", {
    body: "Testo della notifica",
    icon: "/icon.png",
    badge: "/badge.png",
    image: "/preview.png",
    tag: "id-univoco",        // sostituisce notifiche con lo stesso tag
    requireInteraction: false, // resta fino al click se true
    silent: false,
    data: { id: 42 },         // dati personalizzati
    actions: [                // solo in Service Worker
      { action: "apri", title: "Apri" },
      { action: "chiudi", title: "Chiudi" },
    ],
  });

  notifica.addEventListener("click", () => window.focus());
  notifica.addEventListener("close", () => console.log("chiusa"));
  notifica.close(); // chiude dopo 5s
}
```

Web Workers

Eseguono JavaScript in un thread separato (no DOM access).

```
// ── Thread principale ──────────────────────────────

const worker = new Worker("worker.js");

worker.postMessage({ tipo: "calcola", dati: [1, 2, 3] });

worker.addEventListener("message", (event) => {
  console.log("Risultato:", event.data);
});

worker.addEventListener("error", (event) => {
  console.error(event.message, event.filename, event.lineno);
});

worker.terminate(); // termina il worker

// Worker inline (senza file separato)
const blob = new Blob([`
  self.onmessage = (e) => {
    const risultato = e.data * 2;
    self.postMessage(risultato);
  };
`], { type: "application/javascript" });
const workerInline = new Worker(URL.createObjectURL(blob));

// ── worker.js ──────────────────────────────────────

self.addEventListener("message", (event) => {
  const { tipo, dati } = event.data;
  if (tipo === "calcola") {
    const risultato = dati.reduce((a, b) => a + b, 0);
    self.postMessage(risultato);
  }
});

// Importare script nel worker
importScripts("utils.js", "lib.js");

// SharedWorker — condiviso tra più tab
const shared = new SharedWorker("shared-worker.js");
shared.port.postMessage("ciao");
```

```
shared.port.onmessage = (e) => console.log(e.data);
shared.port.start();
```

Console

```
// Output base
console.log("Messaggio normale");
console.info("Informazione");
console.warn("Avvertimento");
console.error("Errore");
console.debug("Debug (nascosto di default)");

// Formattazione
console.log("%s ha %d anni", "Alice", 30);    // stringa, numero
console.log("%c Testo colorato", "color: red; font-size: 1.5em;");
console.log("%o", oggetto);     // rappresentazione interattiva
console.log("%O", oggetto);     // proprietà dell'oggetto

// Tabella
console.table([{ nome: "Alice", eta: 30 }, { nome: "Bob", eta: 25 }]);
console.table({ a: 1, b: 2 });

// Raggruppamento
console.group("Gruppo");
console.log("dentro il gruppo");
console.groupCollapsed("Collassato");
console.log("nascosto di default");
console.groupEnd();
console.groupEnd();

// Timing
console.time("operazione");
// ... codice ...
console.timeLog("operazione"); // stampa il tempo corrente
console.timeEnd("operazione"); // stampa il tempo totale

// Contatore
console.count("click");     // click: 1
console.count("click");     // click: 2
console.countReset("click");

// Stack trace
console.trace("Da dove viene chiamato?");
```

```
// Assert (stampa errore se falso)
console.assert(1 === 1, "Non verrà stampato");
console.assert(1 === 2, "1 non è uguale a 2"); // stampa errore

// Clear
console.clear();
```

Clonazione oggetti

```
// Shallow clone (copia solo il primo livello)
const shallow1 = { ...originale };
const shallow2 = Object.assign({}, originale);

// Deep clone (copia tutti i livelli)
// JSON (perde: Date, undefined, funzioni, Symbol, Map, Set, RegExp)
const deep1 = JSON.parse(JSON.stringify(originale));

// structuredClone (ES2022) — supporta Date, Map, Set, ArrayBuffer, ecc.
const deep2 = structuredClone(originale);

// Cosa NON copia structuredClone: funzioni, Symbol, nodi DOM, prototype pe
const deep3 = structuredClone({ data: new Date(), numeri: new Set([1,2,3])
// { data: Date, numeri: Set } — correttamente clonati

// Con trasferimento (move invece di copy per performance con grandi buffe
const buffer = new ArrayBuffer(1024);
const trasferito = structuredClone({ buffer }, { transfer: [buffer] });
// ora buffer è "neutered" (trasferito, non più usabile)
```

Debounce e throttle

```
// Debounce — esegue solo dopo N ms dall'ultima chiamata
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}

const cercaDebounced = debounce((q) => fetch(`/api?q=${q}`), 300);
input.addEventListener("input", e => cercaDebounced(e.target.value));

// Throttle — esegue al massimo una volta ogni N ms
function throttle(fn, interval) {
  let ultimo = 0;
  return function(...args) {
    const ora = Date.now();
    if (ora - ultimo >= interval) {
      ultimo = ora;
      return fn.apply(this, args);
    }
  };
}

const scrollThrottled = throttle(() => console.log(window.scrollY), 100);
window.addEventListener("scroll", scrollThrottled);

// Debounce con esecuzione immediata (opzionale)
function debounceLeading(fn, delay) {
  let timer;
  let eseguito = false;
  return function(...args) {
    if (!eseguito) { fn.apply(this, args); eseguito = true; }
    clearTimeout(timer);
    timer = setTimeout(() => { eseguito = false; }, delay);
  };
}
```

Currying

Trasforma una funzione con N argomenti in una catena di funzioni ognuna con 1 argomento.

```
// Currying manuale
function somma(a) {
  return function(b) {
    return a + b;
  };
}
const somma5 = somma(5);
somma5(3); // 8
somma(5)(3); // 8

// Utility curry generica
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn.apply(this, args);
    }
    return function(...altri) {
      return curried.apply(this, args.concat(altri));
    };
  };
}

const moltiplica = curry((a, b, c) => a * b * c);
moltiplica(2)(3)(4);   // 24
moltiplica(2, 3)(4);   // 24
moltiplica(2)(3, 4);   // 24
moltiplica(2, 3, 4);   // 24

// Partial application
function partial(fn, ...preArgs) {
  return function(...args) {
    return fn(...preArgs, ...args);
  };
}
const doppio = partial(moltiplica, 2, 1);
doppio(5); // 10
```

Memoization

Cache dei risultati di funzioni pure costose.

```
// Memoization semplice (chiave = primo argomento)
function memoize(fn) {
  const cache = new Map();
  return function(arg) {
    if (cache.has(arg)) return cache.get(arg);
    const risultato = fn.call(this, arg);
    cache.set(arg, risultato);
    return risultato;
  };
}

const fattorialeMemo = memoize(function fatt(n) {
  return n <= 1 ? 1 : n * fatt(n - 1);
});
fattorialeMemo(10); // calcola
fattorialeMemo(10); // dalla cache

// Memoization con chiave composta
function memoizeMulti(fn) {
  const cache = new Map();
  return function(...args) {
    const chiave = JSON.stringify(args);
    if (cache.has(chiave)) return cache.get(chiave);
    const risultato = fn.apply(this, args);
    cache.set(chiave, risultato);
    return risultato;
  };
}

// Con WeakMap (per oggetti — garbage collectable)
function memoizeOggetto(fn) {
  const cache = new WeakMap();
  return function(obj) {
    if (cache.has(obj)) return cache.get(obj);
    const risultato = fn.call(this, obj);
    cache.set(obj, risultato);
    return risultato;
  };
}
```

html javascript